QUESTION ONE

```
#region imports
import sys
import numpy as np
from scipy.integrate import quad
from PyQt5 import QtWidgets as qtw
from matplotlib.backends.backend_qt5agg import FigureCanvasQTAgg
from matplotlib.figure import Figure
#endregion


#region model
class TakeoffModel:
"""
Model class for takeoff distance calculation.
ChatGPT helped write this function.

The model calculates aircraft takeoff distance using:
Vstall, VTO, A, B, and STO integral.
"""

def __init__(self):
self.rho = 0.002377
self.S = 1000.0
self.CLmax = 2.4
self.CD = 0.0279
```

```
self.gc = 32.174

def calc_sto(self, weight, thrust):
"""
Calculate takeoff distance for a given weight and thrust.

:param weight: aircraft weight, lb
:param thrust: engine thrust, lbf
:return: takeoff distance, ft
"""
v_stall = np.sqrt(weight / (0.5 * self.rho * self.S * self.CLmax))
v_to = 1.2 * v_stall

A = self.gc * thrust / weight
B = (self.gc / weight) * (0.5 * self.rho * self.S * self.CD)

def integrand(v):
return v / (A - B * v ** 2)

sto, error = quad(integrand, 0, v_to)
return sto

def generate_curve(self, weight, thrust_values):
"""
Generate STO values over a range of thrust values.
```

:param weight: aircraft weight

:param thrust_values: array of thrust values

:return: array of STO values

"""

return np.array([self.calc_sto(weight, thrust) for thrust in thrust_values])

#endregion

#region view

class TakeoffView:

"""

View class for GUI widgets and plotting.

"""

def __init__(self):

self.window = qtw.QWidget()

self.window.setWindowTitle("Aircraft Takeoff Distance Calculator")

self.main_layout = qtw.QVBoxLayout(self.window)

self.input_group = qtw.QGroupBox("Input Parameters")

self.input_layout = qtw.QGridLayout(self.input_group)

self.lbl_weight = qtw.QLabel("Weight, W (lb):")

self.le_weight = qtw.QLineEdit("56000")

self.lbl_thrust = qtw.QLabel("Thrust, T (lbf):")

self.le_thrust = qtw.QLineEdit("13000")

self.btn_calc = qtw.QPushButton("Calculate and Plot")

self.input_layout.addWidget(self.lbl_weight, 0, 0)

self.input_layout.addWidget(self.le_weight, 0, 1)

self.input_layout.addWidget(self.lbl_thrust, 1, 0)

self.input_layout.addWidget(self.le_thrust, 1, 1)

self.input_layout.addWidget(self.btn_calc, 2, 0, 1, 2)

self.main_layout.addWidget(self.input_group)

self.figure = Figure(figsize=(7, 5), tight_layout=True)

self.canvas = FigureCanvasQTAgg(self.figure)

self.ax = self.figure.add_subplot(111)

self.main_layout.addWidget(self.canvas)

self.output_label = qtw.QLabel("STO result will appear here.")

self.main_layout.addWidget(self.output_label)

def show(self):

self.window.show()

def get_inputs(self):

"""

Read weight and thrust from line edits.

"""

weight = float(self.le_weight.text())

thrust = float(self.le_thrust.text())

return weight, thrust

def plot_results(self, thrust_values, curve_data, selected_thrust, selected_sto):

"""

Plot the three STO curves and mark the selected point.

"""

self.ax.clear()

for label, sto_values in curve_data:

self.ax.plot(thrust_values, sto_values, label=label)

self.ax.plot(

selected_thrust,

selected_sto,

marker="o",

markersize=9,

fillstyle="none",

linestyle="None",

label="Selected W and T"

)

```
self.ax.set_title("Takeoff Distance vs Thrust")
self.ax.set_xlabel("Thrust (lbf)")
self.ax.set_ylabel("Takeoff Distance, STO (ft)")
self.ax.grid(True)
self.ax.legend()

self.canvas.draw()

def update_output(self, sto):
"""
Display calculated STO.
"""
self.output_label.setText(f"Selected takeoff distance STO = {sto:0.2f} ft")
#endregion


#region controller
class TakeoffController:
"""
Controller class connecting the model and view.
"""

def __init__(self):
self.model = TakeoffModel()
self.view = TakeoffView()
```

```
self.view.btn_calc.clicked.connect(self.calculate)

def calculate(self):
"""
Read inputs, calculate STO curves, and update plot.
"""
weight, thrust = self.view.get_inputs()

thrust_values = np.linspace(5000, 30000, 100)

weights = [
weight - 10000,
weight,
weight + 10000
]

curve_data = []

for w in weights:
sto_values = self.model.generate_curve(w, thrust_values)

if w == weight:
label = f"W = {w:0.0f} lb"
elif w < weight:
label = f"W - 10000 = {w:0.0f} lb"
else:
```

label = f"W + 10000 = {w:0.0f} lb"

curve_data.append((label, sto_values))

selected_sto = self.model.calc_sto(weight, thrust)

self.view.plot_results(

thrust_values,

curve_data,

thrust,

selected_sto

)

self.view.update_output(selected_sto)

def show(self):

self.view.show()

#endregion

#region main

if __name__ == "__main__":

app = qtw.QApplication(sys.argv)

controller = TakeoffController()

controller.show()

sys.exit(app.exec())

#endregion

QUESTION 2

```
#region imports
import statistics as stats
import random as rnd
from Polymer import macroMolecule, Position
#endregion


#region helper functions
def get_int_input(prompt, default):
    """
    Gets an integer from the user.
    If the user presses Enter, the default value is used.
    ChatGPT helped write this function.
    """
    user_input = input(prompt)

    if user_input.strip() == "":
        return default

    return int(user_input)


def average_position(positions):
    """
    Calculates the average x, y, and z center of mass.
    """
    x_avg = stats.mean([p.x for p in positions])
    y_avg = stats.mean([p.y for p in positions])
    z_avg = stats.mean([p.z for p in positions])

    return Position(x=x_avg, y=y_avg, z=z_avg)


def std_dev(values):
    """
    Calculates sample standard deviation.
```

Returns 0 if there is only one molecule.
"""
if len(values) < 2:
return 0.0

return stats.stdev(values)
#endregion

#region main simulation
def run_polymer_simulation(target_N=1000, number_molecules=50):
"""
Runs freely jointed chain simulation for many polymer molecules.

The actual degree of polymerization for each molecule is selected from a normal distribution with mean N and standard deviation 0.1N.
"""

centers_of_mass = []
end_to_end_distances = []
radii_of_gyration = []
molecular_weights = []
actual_N_values = []

for i in range(number_molecules):
# Required by problem statement:
# N is selected from normal distribution with mean=N and std=0.1N.
actual_N = int(round(rnd.gauss(target_N, 0.1 * target_N)))

if actual_N < 1:
actual_N = 1

polymer = macroMolecule(degreeOfPolymerization=actual_N)
polymer.freelyJointedChainModel()

centers_of_mass.append(polymer.centerOfMass)
end_to_end_distances.append(polymer.endToEndDistance)
radii_of_gyration.append(polymer.radiusOfGyration)

```
molecular_weights.append(polymer.MW)
actual_N_values.append(actual_N)

avg_com = average_position(centers_of_mass)

# Convert from meters to micrometers
end_to_end_um = [value * 1e6 for value in end_to_end_distances]
radius_gyration_um = [value * 1e6 for value in radii_of_gyration]

# Convert from meters to nanometers
avg_com_nm = Position(
    x=avg_com.x * 1e9,
    y=avg_com.y * 1e9,
    z=avg_com.z * 1e9
)

# PDI = Mw / Mn
Mn = stats.mean(molecular_weights)
Mw = sum([mw ** 2 for mw in molecular_weights]) / sum(molecular_weights)
PDI = Mw / Mn

results = {
    "avg_com_nm": avg_com_nm,
    "end_avg_um": stats.mean(end_to_end_um),
    "end_std_um": std_dev(end_to_end_um),
    "rg_avg_um": stats.mean(radius_gyration_um),
    "rg_std_um": std_dev(radius_gyration_um),
    "PDI": PDI,
    "actual_N_avg": stats.mean(actual_N_values)
}

return results
#endregion


#region output
def print_results(target_N, number_molecules, results):
    """
```

```
    *Prints polymer statistics in the format requested*.
    """

    com = results["avg_com_nm"]

    print(f"Metrics for {number_molecules} molecules of degree of polymerization = {target_N}")
    print(f"Avg. Center of Mass (nm) = {com.x:0.3f}, {com.y:0.3f}, {com.z:0.3f}")

    print("End-to-end distance (μm):")
    print(f"   Average = {results['end_avg_um']:0.3f}")
    print(f"   Std. Dev. = {results['end_std_um']:0.3f}")

    print("Radius of gyration (μm):")
    print(f"   Average = {results['rg_avg_um']:0.3f}")
    print(f"   Std. Dev. = {results['rg_std_um']:0.3f}")

    print(f"PDI = {results['PDI']:0.3f}")
    #endregion


    #region main
    def main():
        """
        *Main CLI function*.
        """

        target_N = get_int_input("degree of polymerization (1000)?: ", 1000)
        number_molecules = get_int_input("How many molecules (50)?: ", 50)

        results = run_polymer_simulation(target_N, number_molecules)
        print_results(target_N, number_molecules, results)


    if __name__ == "__main__":
        main()
    #endregion
```

QUESTION THREE

```
#region imports
from OttoDiesel_GUI import Ui_Form
import sys
from PyQt5 import QtWidgets as qtw
from Otto import ottoCycleController
from Diesel import dieselCycleController

from matplotlib.backends.backend_qt5agg import FigureCanvasQTAgg
from matplotlib.figure import Figure
#endregion


class MainWindow(qtw.QWidget, Ui_Form):
    def __init__(self):
        """MainWindow constructor."""
        super().__init__()
        self.setupUi(self)
        self.calculated = False

        # Create matplotlib canvas
        self.figure = Figure(figsize=(8, 8), tight_layout=True, frameon=True, facecolor='none')
        self.canvas = FigureCanvasQTAgg(self.figure)
        self.ax = self.figure.add_subplot()
        self.main_VerticalLayout.addWidget(self.canvas)

        # Signals and slots
        self.rdo_Metric.toggled.connect(self.setUnits)
        self.btn_Calculate.clicked.connect(self.calcCycle)
        self.cmb_Abcissa.currentIndexChanged.connect(self.doPlot)
        self.cmb_Ordinate.currentIndexChanged.connect(self.doPlot)
        self.chk_LogAbcissa.stateChanged.connect(self.doPlot)
        self.chk_LogOrdinate.stateChanged.connect(self.doPlot)
        self.cmb_OttoDiesel.currentIndexChanged.connect(self.selectCycle)

        # ChatGPT helped complete these controller connections.
        self.otto = ottoCycleController(ax=self.ax)
```

```
self.diesel = dieselCycleController(ax=self.ax)
self.controller = self.otto

self.someWidgets = []
self.someWidgets += [self.lbl_THigh, self.lbl_TLow, self.lbl_P0, self.lbl_V0, self.lbl_CR]
self.someWidgets += [self.le_THigh, self.le_TLow, self.le_P0, self.le_V0, self.le_CR]
self.someWidgets += [self.le_T1, self.le_T2, self.le_T3, self.le_T4]
self.someWidgets += [self.lbl_T1Units, self.lbl_T2Units, self.lbl_T3Units,
self.lbl_T4Units]
self.someWidgets += [self.le_PowerStroke, self.le_CompressionStroke,
self.le_HeatAdded, self.le_Efficiency]
self.someWidgets += [self.lbl_PowerStrokeUnits, self.lbl_CompressionStrokeUnits,
self.lbl_HeatInUnits]
self.someWidgets += [self.rdo_Metric, self.cmb_Abcissa, self.cmb_Ordinate]
self.someWidgets += [self.chk_LogAbcissa, self.chk_LogOrdinate, self.ax, self.canvas]

self.otto.setWidgets(w=self.someWidgets)
self.diesel.setWidgets(w=self.someWidgets)

self.selectCycle()
self.show()

def clamp(self, val, low, high):
"""Clamp a numeric value between low and high."""
if self.isfloat(val):
val = float(val)
if val > high:
return float(high)
if val < low:
return float(low)
return val
return float(low)

def isfloat(self, value):
"""Check whether a string can be converted to float."""
if value == 'NaN':
return False
try:
```

```
float(value)
return True
except ValueError:
return False

def doPlot(self):
"""Update plot when axis or log options change."""
self.controller.updateView()

def selectCycle(self):
"""Select Otto or Diesel cycle controller."""
otto = self.cmb_OttoDiesel.currentIndex() == 0
self.gb_Input.setTitle(
'Input for Air Standard {} Cycle:'.format('Otto' if otto else 'Diesel')
)
self.controller = self.otto if otto else self.diesel
self.controller.updateView()

def setUnits(self):
"""Update view when units are changed."""
self.controller.updateView()

def calcCycle(self):
"""Calculate selected thermodynamic cycle."""
self.controller.calc()
self.calculated = True


if __name__ == '__main__':
app = qtw.QApplication(sys.argv)
mw = MainWindow()
mw.setWindowTitle('Otto/Diesel Cycle Calculator')
sys.exit(app.exec())
```

QUESTION FOUR

```
#region imports
from scipy.integrate import odeint
from scipy.optimize import minimize
import matplotlib.pyplot as plt
import numpy as np
import math
from PyQt5 import QtWidgets as qtw
from PyQt5 import QtCore as qtc
from PyQt5 import QtGui as qtg

# These imports are necessary for drawing a matplotlib graph on the GUI.
from matplotlib.backends.backend_qt5agg import FigureCanvasQTAgg
from matplotlib.figure import Figure
#endregion

#region class definitions
#region specialized graphic items
class MassBlock(qtw.QGraphicsItem):
def __init__(self, CenterX, CenterY, width=30, height=10, parent=None, pen=None,
brush=None, name='CarBody', mass=10):
"""
*Rectangular mass block used in the quarter-car schematic*.
"""
super().__init__(parent)
self.x = CenterX
self.y = CenterY
self.pen = pen
self.brush = brush
self.width = width
self.height = height
self.top = self.y - self.height / 2
self.left = self.x - self.width / 2
self.rect = qtc.QRectF(self.left, self.top, self.width, self.height)
self.name = name
self.mass = mass
```

```
self.transformation = qtg.QTransform()
stTT = self.name + "\nx={:0.3f}, y={:0.3f}\nmass = {:0.3f}".format(self.x, self.y, self.mass)
self.setToolTip(stTT)

def boundingRect(self):
"""Return bounding rectangle for the transformed graphics item."""
return self.transformation.mapRect(self.rect)

def paint(self, painter, option, widget=None):
"""Paint the rectangular mass block."""
self.transformation.reset()
if self.pen is not None:
 painter.setPen(self.pen)
if self.brush is not None:
painter.setBrush(self.brush)
self.top = -self.height / 2
self.left = -self.width / 2
self.rect = qtc.QRectF(self.left, self.top, self.width, self.height)
painter.drawRect(self.rect)
self.transformation.translate(self.x, self.y)
self.setTransform(self.transformation)
self.transformation.reset()


class Wheel(qtw.QGraphicsItem):
def __init__(self, CenterX, CenterY, radius=10, parent=None, pen=None,
wheelBrush=None, massBrush=None, name='Wheel', mass=10):
"""
Wheel item used in the quarter-car schematic.
"""
super().__init__(parent)
self.x = CenterX
self.y = CenterY
 self.pen = pen
self.brush = wheelBrush
self.radius = radius
self.rect = qtc.QRectF(self.x - self.radius, self.y - self.radius, self.radius * 2, self.radius *
2)
```

self.name = name
self.mass = mass
self.transformation = qtg.QTransform()
stTT = self.name + "\nx={:0.3f}, y={:0.3f}\nmass = {:0.3f}".format(self.x, self.y, self.mass)
self.setToolTip(stTT)
self.massBlock = MassBlock(CenterX, CenterY, width=2 * radius * 0.85, height=radius / 3,
pen=pen, brush=massBrush, name="Wheel Mass", mass=mass)

def boundingRect(self):
"""Return bounding rectangle for the transformed graphics item."""
return self.transformation.mapRect(self.rect)

def addToScene(self, scene):
"""Add the wheel and its small mass block to the QGraphicsScene."""
scene.addItem(self)
scene.addItem(self.massBlock)

def paint(self, painter, option, widget=None):
"""Paint the wheel."""
self.transformation.reset()
if self.pen is not None:
painter.setPen(self.pen)
if self.brush is not None:
painter.setBrush(self.brush)
self.rect = qtc.QRectF(-self.radius, -self.radius, self.radius * 2, self.radius * 2)
painter.drawEllipse(self.rect)
self.transformation.translate(self.x, self.y)
self.setTransform(self.transformation)
self.transformation.reset()
#endregion

#region MVC for quarter car model
class CarModel():
"""
Model class for the quarter-car suspension system.

Stores vehicle parameters, simulation results, road/ramp information,

```
    spring limits, acceleration data, and the SSE objective value.
    """
    def __init__(self):
        """
        Construct the quarter-car model with reasonable default values.
        ChatGPT helped complete this constructor.
        """
        self.results = None
        self.tmax = 3.0
        self.t = np.linspace(0, self.tmax, 200)
        self.tramp = 1.0
        self.angrad = 0.1
        self.ymag = 6.0 / (12.0 * 3.3)  # approximate 6 in in meters
        self.yangdeg = 45.0

        # Default car properties.
        self.m1 = 450.0     # car body mass in kg, quarter-car mass
        self.m2 = 20.0      # wheel mass in kg
        self.c1 = 4500.0    # damping coefficient in N*s/m
        self.k1 = 15000.0   # suspension spring constant in N/m
        self.k2 = 90000.0   # tire spring constant in N/m
        self.v = 120.0      # vehicle speed in kph

        # Static-compression based spring bounds.
        self.updateSpringLimits()

        self.accel = None
        self.accelMax = 0.0
        self.accelLim = 2.0  # passenger acceleration limit in g's
        self.SSE = 0.0

    def updateSpringLimits(self):
        """
        Calculate k1 and k2 lower/upper limits from static compression assumptions.

        Suspension compression range: 3 in to 6 in.
        Tire compression range: 0.75 in to 1.5 in.
        """
```

```
g = 9.81
dx1_min = 3.0 * 0.0254
dx1_max = 6.0 * 0.0254
dx2_min = 0.75 * 0.0254
dx2_max = 1.5 * 0.0254

self.mink1 = self.m1 * g / dx1_max
self.maxk1 = self.m1 * g / dx1_min
self.mink2 = (self.m1 + self.m2) * g / dx2_max
self.maxk2 = (self.m1 + self.m2) * g / dx2_min


class CarView():
def __init__(self, args):
"""
View class for quarter-car GUI display widgets and plotting.
"""
self.input_widgets, self.display_widgets = args
self.le_m1, self.le_v, self.le_k1, self.le_c1, self.le_m2, self.le_k2, self.le_ang, \
self.le_tmax, self.chk_IncludeAccel = self.input_widgets

self.gv_Schematic, self.chk_LogX, self.chk_LogY, self.chk_LogAccel, \
self.chk_ShowAccel, self.lbl_MaxMinInfo, self.layout_horizontal_main =
self.display_widgets

self.figure = Figure(tight_layout=True, frameon=True, facecolor='none')
self.canvas = FigureCanvasQTAgg(self.figure)
self.layout_horizontal_main.addWidget(self.canvas)

self.ax = self.figure.add_subplot()
self.ax1 = self.ax.twinx() if self.ax is not None else None

self.buildScene()

def updateView(self, model=None):
"""Update GUI line edits, labels, and plot from the model."""
self.le_m1.setText("{:0.2f}".format(model.m1))
self.le_k1.setText("{:0.2f}".format(model.k1))
```

```
self.le_c1.setText("{:0.2f}".format(model.c1))
self.le_m2.setText("{:0.2f}".format(model.m2))
self.le_k2.setText("{:0.2f}".format(model.k2))
self.le_ang.setText("{:0.2f}".format(model.yangdeg))
self.le_tmax.setText("{:0.2f}".format(model.tmax))
stTmp = "k1_min = {:0.2f}, k1_max = {:0.2f}\nk2_min = {:0.2f}, k2_max = {:0.2f}\n".format(
model.mink1, model.maxk1, model.mink2, model.maxk2)
stTmp += "SSE = {:0.4f}\nMax Accel = {:0.3f} g".format(model.SSE, model.accelMax)
self.lbl_MaxMinInfo.setText(stTmp)
self.doPlot(model)

def buildScene(self):
"""Build a simple quarter-car schematic in the graphics view."""
self.scene = qtw.QGraphicsScene()
self.scene.setObjectName("MyScene")
self.scene.setSceneRect(-200, -200, 400, 400)
self.gv_Schematic.setScene(self.scene)
self.setupPensAndBrushes()

self.Wheel = Wheel(0, 50, 50, pen=self.penWheel, wheelBrush=self.brushWheel,
massBrush=self.brushMass, name="Wheel")
self.CarBody = MassBlock(0, -70, 100, 30, pen=self.penWheel, brush=self.brushMass,
name="Car Body", mass=150)
self.Wheel.addToScene(self.scene)
self.scene.addItem(self.CarBody)

# Simple schematic lines: suspension, tire, and road.
self.scene.addLine(0, -55, 0, 0, self.penSuspension)
self.scene.addLine(-20, -45, 20, -35, self.penSuspension)
self.scene.addLine(20, -35, -20, -25, self.penSuspension)
self.scene.addLine(-20, -25, 20, -15, self.penSuspension)
self.scene.addLine(20, -15, 0, 0, self.penSuspension)
self.scene.addLine(0, 0, 0, 50, self.penTire)
self.scene.addLine(-120, 105, 120, 105, self.penRoad)
self.scene.addText("Quarter Car Model")

def setupPensAndBrushes(self):
"""Set pens and brushes for the graphics scene."""
```

```
self.penWheel = qtg.QPen(qtg.QColor("orange"))
self.penWheel.setWidth(2)
self.penSuspension = qtg.QPen(qtg.QColor("blue"))
self.penSuspension.setWidth(2)
self.penTire = qtg.QPen(qtg.QColor("darkGreen"))
self.penTire.setWidth(2)
self.penRoad = qtg.QPen(qtg.QColor("black"))
self.penRoad.setWidth(2)
self.brushWheel = qtg.QBrush(qtg.QColor.fromHsv(35, 255, 255, 64))
self.brushMass = qtg.QBrush(qtg.QColor(200, 200, 200, 128))

def roadProfile(self, model):
"""Return road y-position data for plotting."""
road = np.zeros_like(model.t)
for i, ti in enumerate(model.t):
if ti < model.tramp:
road[i] = model.ymag * (ti / model.tramp)
else:
road[i] = model.ymag
return road

def doPlot(self, model=None):
"""Plot car body position, wheel position, road contour, and optional acceleration."""
if model is None or model.results is None:
return

ax = self.ax
ax1 = self.ax1
QTPlotting = True
if ax is None:
ax = plt.subplot()
ax1 = ax.twinx()
QTPlotting = False

ax.clear()
ax1.clear()

t = model.t
```

```
ycar = model.results[:, 0]
ywheel = model.results[:, 2]
accel = model.accel if model.accel is not None else np.zeros_like(t)
yroad = self.roadProfile(model)

if self.chk_LogX.isChecked():
ax.set_xlim(0.001, model.tmax)
ax.set_xscale('log')
else:
ax.set_xlim(0.0, model.tmax)
ax.set_xscale('linear')

y_max = max(float(np.max(ycar)), float(np.max(ywheel)), float(np.max(yroad)),
model.ymag) * 1.10
y_max = max(y_max, 0.001)
if self.chk_LogY.isChecked():
ax.set_ylim(0.0001, y_max)
ax.set_yscale('log')
else:
ax.set_ylim(0.0, y_max)
ax.set_yscale('linear')

ax.plot(t, ycar, 'b-', label='Body Position')
ax.plot(t, ywheel, 'r-', label='Wheel Position')
ax.plot(t, yroad, 'k--', label='Road Profile')

if self.chk_ShowAccel.isChecked():
ax1.plot(t, accel, 'g-', label='Body Accel')
ax1.axhline(y=model.accelLim, color='orange')
ax1.axhline(y=-model.accelLim, color='orange')
ax1.set_yscale('log' if self.chk_LogAccel.isChecked() else 'linear')

ax.set_ylabel("Vertical Position (m)", fontsize='large' if QTPlotting else 'medium')
ax.set_xlabel("time (s)", fontsize='large' if QTPlotting else 'medium')
ax1.set_ylabel("Y'' (g)", fontsize='large' if QTPlotting else 'medium')
ax.legend(loc='best')

if self.chk_ShowAccel.isChecked():
```

```
ax1.legend(loc='upper right')

ax.axvline(x=model.tramp)
ax.axhline(y=model.ymag)
ax.tick_params(axis='both', which='both', direction='in', top=True,
               labelsize='large' if QTPlotting else 'medium')
ax1.tick_params(axis='both', which='both', direction='in', right=True,
               labelsize='large' if QTPlotting else 'medium')

if not QTPlotting:
    plt.show()
else:
    self.canvas.draw()


class CarController():
    def __init__(self, args):
        """
        Controller class for the quarter-car model.

        Connects GUI widgets to model calculations and view plotting.
        """
        self.input_widgets, self.display_widgets = args
        self.le_m1, self.le_v, self.le_k1, self.le_c1, self.le_m2, self.le_k2, self.le_ang, \
        self.le_tmax, self.chk_IncludeAccel = self.input_widgets

        self.gv_Schematic, self.chk_LogX, self.chk_LogY, self.chk_LogAccel, \
        self.chk_ShowAccel, self.lbl_MaxMinInfo, self.layout_horizontal_main = \
        self.display_widgets

        self.model = CarModel()
        self.view = CarView(args)
        self.view.updateView(self.model)

    def ode_system(self, X, t):
        """
        Differential equation system for the quarter-car model.
```

X = [x1, x1dot, x2, x2dot]
x1 = car body vertical position
x2 = wheel hub vertical position
"""
if t < self.model.tramp:
y = self.model.ymag * (t / self.model.tramp)
else:
y = self.model.ymag

x1 = X[0]
x1dot = X[1]
x2 = X[2]
x2dot = X[3]

# ChatGPT helped complete the quarter-car ODE equations.
x1ddot = (-self.model.k1 * (x1 - x2) - self.model.c1 * (x1dot - x2dot)) / self.model.m1
x2ddot = (self.model.k1 * (x1 - x2) + self.model.c1 * (x1dot - x2dot)
- self.model.k2 * (x2 - y)) / self.model.m2

return [x1dot, x1ddot, x2dot, x2ddot]

def read_float(self, widget, default):
"""Read a float from a QLineEdit, falling back to default if needed."""
try:
return float(widget.text())
except Exception:
return default

def calculate(self, doCalc=True):
"""
Read GUI inputs, update model parameters, solve the ODEs, calculate SSE, and update the view.
"""
self.model.m1 = self.read_float(self.le_m1, self.model.m1)
self.model.m2 = self.read_float(self.le_m2, self.model.m2)
self.model.c1 = self.read_float(self.le_c1, self.model.c1)
self.model.k1 = self.read_float(self.le_k1, self.model.k1)
self.model.k2 = self.read_float(self.le_k2, self.model.k2)

self.model.v = self.read_float(self.le_v, self.model.v)
self.model.yangdeg = self.read_float(self.le_ang, self.model.yangdeg)
self.model.tmax = self.read_float(self.le_tmax, self.model.tmax)

self.model.updateSpringLimits()

# 6 inch ramp height converted approximately to meters.
self.model.ymag = 6.0 / (12.0 * 3.3)

if doCalc:
self.doCalc()
else:
# Still calculate once so SSE and acceleration are valid for optimization.
self.doCalc(doPlot=False)

self.SSE((self.model.k1, self.model.c1, self.model.k2), optimizing=False)
self.view.updateView(self.model)

def doCalc(self, doPlot=True, doAccel=True):
"""
Solve the quarter-car ODE system and optionally update plot and acceleration.
"""
v = max(1000.0 * self.model.v / 3600.0, 1.0e-9)
self.model.angrad = self.model.yangdeg * math.pi / 180.0
denom = max(abs(math.sin(self.model.angrad)) * v, 1.0e-9)
self.model.tramp = self.model.ymag / denom

self.model.t = np.linspace(0, self.model.tmax, 2000)
ic = [0.0, 0.0, 0.0, 0.0]
self.model.results = odeint(self.ode_system, ic, self.model.t)

if doAccel:
self.calcAccel()
if doPlot:
self.doPlot()

def calcAccel(self):
"""*Calculate body acceleration in g's from body velocity using finite differences.*"""

```
N = len(self.model.t)
self.model.accel = np.zeros(shape=N)
vel = self.model.results[:, 1]
for i in range(N):
if i == N - 1:
h = self.model.t[i] - self.model.t[i - 1]
self.model.accel[i] = (vel[i] - vel[i - 1]) / (9.81 * h)
else:
h = self.model.t[i + 1] - self.model.t[i]
self.model.accel[i] = (vel[i + 1] - vel[i]) / (9.81 * h)
self.model.accelMax = float(np.max(np.abs(self.model.accel)))
return True

def OptimizeSuspension(self):
"""
Optimize k1, c1, and k2 using Nelder-Mead to minimize SSE.
"""
# ChatGPT helped complete this optimization workflow.
self.calculate(doCalc=False)
x0 = np.array([self.model.k1, self.model.c1, self.model.k2])

answer = minimize(
self.SSE,
x0,
method='Nelder-Mead',
options={'maxiter': 300, 'disp': True}
)

self.model.k1, self.model.c1, self.model.k2 = answer.x
self.doCalc(doPlot=True)
self.SSE((self.model.k1, self.model.c1, self.model.k2), optimizing=False)
self.view.updateView(self.model)

def SSE(self, vals, optimizing=True):
"""
Calculate sum of squared errors between car body position and road contour.

Penalties are added for spring limits, damping limit, and optional acceleration limit.
```

```
"""
k1, c1, k2 = vals
self.model.k1 = float(k1)
self.model.c1 = float(c1)
self.model.k2 = float(k2)

self.doCalc(doPlot=False)

SSE = 0.0
for i in range(len(self.model.results[:, 0])):
t = self.model.t[i]
y = self.model.results[:, 0][i]
if t < self.model.tramp:
ytarget = self.model.ymag * (t / self.model.tramp)
else:
ytarget = self.model.ymag
SSE += (y - ytarget) ** 2

if optimizing:
if k1 < self.model.mink1 or k1 > self.model.maxk1:
SSE += 1000000.0
if c1 < 10.0:
SSE += 1000000.0
if k2 < self.model.mink2 or k2 > self.model.maxk2:
SSE += 1000000.0

if self.model.accelMax > self.model.accelLim and
self.chk_IncludeAccel.isChecked():
SSE += 1000000.0 * (self.model.accelMax - self.model.accelLim) ** 2

self.model.SSE = float(SSE)
return float(SSE)

def doPlot(self):
"""Update the quarter-car plot."""
self.view.doPlot(self.model)
#endregion
#endregion
```

```
def main():
    """
    This model file is intended to be imported by Car_app.py.
    Run Car_app.py to start the GUI.
    """
    print("QuarterCarModel.py is a model/controller module. Run Car_app.py to start the GUI.")


if __name__ == '__main__':
    main()
```

QUESTION 5

```
#region imports
from Car_GUI import Ui_Form
import sys
from PyQt5 import QtCore as qtc
from PyQt5 import QtWidgets as qtw
from PyQt5 import QtGui as qtg
from QuarterCarModel import CarController

#these imports are necessary for drawing a matplot lib graph on my GUI
#no simple widget for this exists in QT Designer, so I have to add the widget in code.
from matplotlib.backends.backend_qt5agg import FigureCanvasQTAgg
from matplotlib.figure import Figure
#endregion

class MainWindow(qtw.QWidget, Ui_Form):
    def __init__(self):
        """
        Main window constructor.
        """
        super().__init__()
        #call setupUi feom Ui_Form parent
        self.setupUi(self)

        #setup car controller
        input_widgets = (self.le_m1, self.le_v, self.le_k1, self.le_c1, self.le_m2, self.le_k2,
                         self.le_ang, \
                         self.le_tmax, self.chk_IncludeAccel)
        display_widgets = (self.gv_Schematic, self.chk_LogX, self.chk_LogY, self.chk_LogAccel,
                           \
                           self.chk_ShowAccel, self.lbl_MaxMinInfo, self.layout_horizontal_main)

        #instantiate the car controller
        self.controller = CarController((input_widgets, display_widgets))

        # connect signal to slots
        self.btn_calculate.clicked.connect(self.controller.calculate)
```

```
self.pb_Optimize.clicked.connect(self.doOptimize)
self.chk_LogX.stateChanged.connect(self.controller.doPlot)
self.chk_LogY.stateChanged.connect(self.controller.doPlot)
self.chk_LogAccel.stateChanged.connect(self.controller.doPlot)
self.chk_ShowAccel.stateChanged.connect(self.controller.doPlot)
self.show()

def doOptimize(self):
app.setOverrideCursor(qtc.Qt.WaitCursor)
self.controller.OptimizeSuspension()
app.restoreOverrideCursor()

if __name__ == '__main__':
app = qtw.QApplication(sys.argv)
mw = MainWindow()
mw.setWindowTitle('Quarter Car Model')
sys.exit(app.exec())
```

QUESTION 6

```
#region imports
from Truss_GUI import Ui_TrussStructuralDesign
from PyQt5 import QtWidgets as qtw
from PyQt5 import QtCore as qtc
from Truss_Classes import TrussController
import sys
#endregion


#region class definitions
class MainWindow(Ui_TrussStructuralDesign, qtw.QWidget):
    def __init__(self):
        """
        Main GUI window for the truss design program.
        The App only communicates directly with the Controller.
        """
        super().__init__()
        self.setupUi(self)

        self.btn_Open.clicked.connect(self.OpenFile)
        self.spnd_Zoom.valueChanged.connect(self.setZoom)

        self.controller = TrussController()

        self.controller.setDisplayWidgets((
            self.te_DesignReport,
            self.le_LinkName,
            self.le_Node1Name,
            self.le_Node2Name,
            self.le_LinkLength,
            self.gv_Main
            ))

        # MVC fix: App asks controller to install scene event filter
        self.controller.installSceneEventFilter(self)
```

self.gv_Main.setMouseTracking(True)

self.show()

def setZoom(self):
"""
Updates graphics view zoom.
"""
self.gv_Main.resetTransform()
self.gv_Main.scale(self.spnd_Zoom.value(), self.spnd_Zoom.value())

def eventFilter(self, obj, event):
"""
Handles mouse movement and mouse wheel zoom for the graphics scene.
The App does not directly access controller.view.
"""

if self.controller.isSceneObject(obj):

if event.type() == qtc.QEvent.GraphicsSceneMouseMove:
scenePos = event.scenePos()

strScene = "Mouse Position: x = {}, y = {}".format(
round(scenePos.x(), 2),
round(-scenePos.y(), 2)
)

item = self.controller.getItemAt(
scenePos,
self.gv_Main.transform()
)

if item is not None and item.data(0) is not None:
strScene += " (" + item.data(0) + ")"

items = self.controller.getItemsAt(scenePos)

item_names = [

```
                item.name if hasattr(item, "name") else None
                for item in items
                ]

                for name in item_names:
                strScene += ", " + (name if name is not None else "none")

                self.lbl_MousePos.setText(strScene)

            elif event.type() == qtc.QEvent.GraphicsSceneWheel:
                # PyQt5 safer wheel event syntax
                if event.angleDelta().y() > 0:
                    self.spnd_Zoom.stepUp()
                else:
                self.spnd_Zoom.stepDown()

                return True

        return super(MainWindow, self).eventFilter(obj, event)

    def OpenFile(self):
        """
        Opens a truss input file and passes the file data to the controller.
        """
        filename = qtw.QFileDialog.getOpenFileName()[0]

        if len(filename) == 0:
            return

        self.te_Path.setText(filename)

        with open(filename, "r") as file:
            data = file.readlines()

        self.controller.ImportFromFile(data)
        #endregion
```

```
#region function definitions
def Main():
app = qtw.QApplication(sys.argv)
mw = MainWindow()
sys.exit(app.exec())
#endregion


#region function calls
if __name__ == "__main__":
Main()
#endregion
```